ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΗΛΕΚΤΡΟΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

ΤΟΜΕΑΣ ΤΕΧΝΟΛΟΓΙΑΣ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ

ΕΡΓΑΣΤΗΡΙΟ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

ΣΥΣΤΗΜΑΤΑ ΜΙΚΡΟΫΠΟΛΟΓΙΣΤΩΝ

Αναφορά για την 3η Ομάδα Ασκήσεων

Μέλη Ομάδας

Κωνσταντόπουλος Γεώργιος (03122104)

Πουλόπουλος Ηλίας (03122160)

ΖΩΓΡΑΦΟΥ ΜΑΪΟΣ 2025

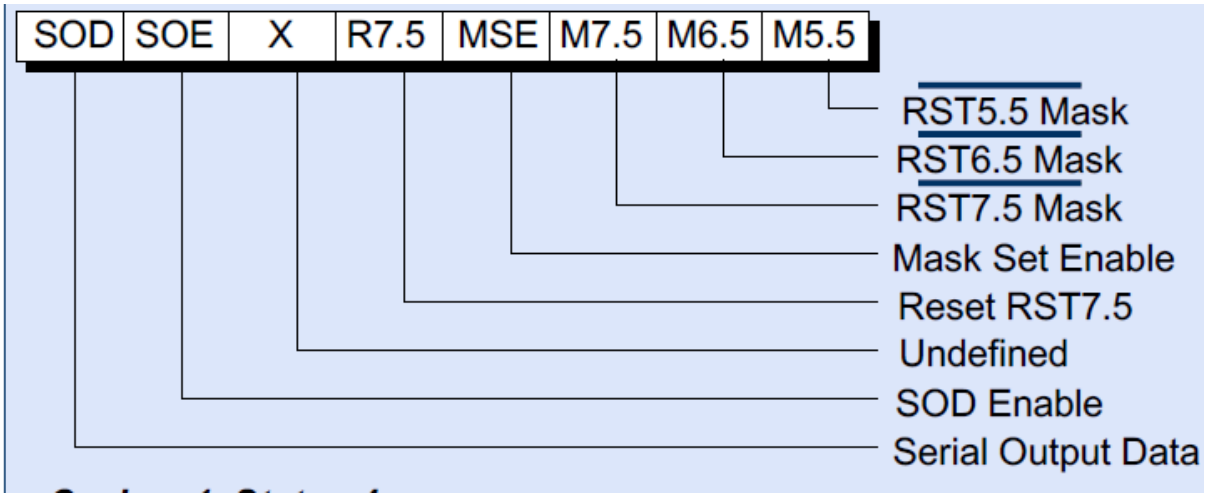
Ασκήσεις Προσομοίωσης

1η Άσκηση

Ο πλήρης κώδικας με κατάλληλα σχόλια για επεξήγηση μπορεί να βρεθεί στο αρχείο ***ex3.1.8085***.

2η Άσκηση

Αρχικά θα πρέπει να θέσουμε τη μάσκα για να μπορεί το πρόγραμμα να δεχθεί μόνο το RST 6.5 interrupt, για αυτό θα πρέπει βάσει της παρακάτω εικόνας να θέσουμε το bit 1 του A ίσο με 0 για να μην κάνουμε mask το RST 6.5 και να θέσουμε όλα τα υπόλοιπα 1 ώστε να αγνοούνται τα υπόλοιπα interrupts.



Επίσης, θέτουμε τα K1=105 και K2=221 (τυχαία) στους καταχωρητές D και E αντίστοιχα.

Θα χρειαστεί ακόμα να αρχικοποιήσουμε την 7-segment οθόνη που λαμβάνει δεδομένα από τις θέσεις μνήμης 0BF0H - 0BF5H θέτοντας και τα 6 bits αυτών των θέσεων ίσα με 10H που αντιστοιχεί στο κενό βάσει της DCD. Για αυτό τον σκοπό θα χρειαστούμε έναν counter των 6 bits που θα μπει στον συσσωρευτή B.

Για αυτό γράφουμε τις παρακάτω εντολές:

```
START:
IN 10H
MVI A,0DH ;A=0000 1101 RST 6.5 Non-Masked
SIM
EI ;Enable Interrupts
MVI D,69H ;K1=105
MVI E,DDH ;K2=221
MVI B,06H ;Counter needed to make all 6 digits of the 7-segment empty
LXI H,0BF0H ;0BF0-0BF5 used by DCD (right to left)

CLEAR:
MVI M,10H ;10H means empty for the display
INX H
DCR B
JNZ CLEAR

WAIT:
JMP WAIT ;Waiting for the interrupt loop
```

Στη συνέχεια θα γράψουμε την ρουτίνα εξυπηρέτησης της διακοπής, η οποία θα καλεί την KIND για να λάβει στον συσσωρευτή A ένα input από το πληκτρολόγιο. Πρώτα διαβάζουμε το λιγότερο σημαντικό ψηφίο του διψήφιου δεκαεξαδικού αριθμού, τον οποίο θα εμφανίσουμε στην 2η από τα αριστερά θέση (OBF4H) όπως φαίνεται και από τον παρακάτω πίνακα:

Διεύθυνση RAM	Ετικέτα	Κωδικοποιημένο ψηφίο (δεκαεξαδικός κώδικας)
0BF0	UDSP0	0 δεξιότερο ψηφίο
0BF1	UDSP1	1
0BF2	UDSP2	2
0BF3	UDSP3	3
0BF4	UDSP4	4
0BF5	UDSP5	5 αριστερότερο ψηφίο

Θα κρατήσουμε αυτό το ψηφίο προσωρινά στον B για να πάρουμε input και το δεύτερο ψηφίο (το σημαντικότερο ψηφίο) στον A. Για να συγκρίνουμε το ολοκληρωμένο input με τους αριθμούς K1, K2 θα πρέπει να κάνουμε 4 περιστροφές

τον A και να προσθέσουμε σε αυτόν το λιγότερο σημαντικό ψηφίο για να έχουμε στον A το ολοκληρωμένο input. Η ρουτίνα εξυπηρέτησης είναι η εξής:

```
INTR_ROUTINE:
;Input Least Significant Digit (LSD)
CALL KIND
STA 0BF4H ;Store in 0BF4 to show at the 2nd most left digit of the display
MOV B,A ;Keep LSD. B=0000 LLLL

;Input Most Significant Digit (MSD)
CALL KIND
STA 0BF5H ;Store in 0BF5 to show at the most left digit of the display
RRC ;Rotate 4 times so that the MSD's bits come to the front
RRC ;A before rotation=0000 MMMM
RRC ;A after rotation=MMMM 0000
RRC
ADD B ;A=MMMM LLLL
```

Στη συνέχεια, συγκρίνουμε με τα K1,K2 και ανάβουμε το LED 0 όταν το input είναι στο [0,K1], το LED 1 όταν είναι στο (K1,K2] και το LED 2 όταν είναι στο (K2,FFH]:

```
;Comparisons
;A<=K1?
CMP D
JC BELOW_K1
JZ BELOW_K1

;A<=K2?
CMP E
JC BETWEEN
JZ BETWEEN

ABOVE_K2:
MVI A,FBH ;A=1111 1011 LED 2 ON when in (K2,FF]
JMP LEDS

BETWEEN:
MVI A,FDH ;A=1111 1101 LED 1 ON when in (K1,K2]
JMP LEDS

BELOW_K1:
MVI A,FEH ;A=1111 1110 LED 0 ON when in [0,K1]
```

```
JMP LEDS

LEDS:
STA 3000H
```

Τέλος, εκτυπώνουμε τον αριθμό στο 7-segment display χρήσει της STDM και DCD που χρησιμοποιούν τον συσσωρευτή D, που σημαίνει ότι πρέπει να κάνουμε PUSH στο stack τον D (που στη συγκεκριμένη περίπτωση περιέχει το K1) και να το επαναφέρουμε μετά την εκτύπωση με ένα POP. Στο D προσωρινά φορτώνουμε τη θέση μνήμης 0BF0H την οποία θα διαβάσει μαζί με τις επόμενες 5 θέσεις μνήμης και θα εκτυπώσει τους αριθμούς που βρίσκονται στις θέσεις 0BF4 και 0BF5 (καθώς οι υπόλοιπες περιέχουν το κενό 10H). Έπειτα καλούμε συνεχώς την DCD για να εκτυπώνεται συνεχώς αυτό το input έως το επόμενο interrupt για να μην τελειώνει το πρόγραμμα:

```
;Print the number on the display
PUSH D
LXI D,0BF0H
CALL STDM
POP D
EI

DISPLAY:
;Keep printing till new interrupt
CALL DCD
JMP DISPLAY

END
```

Βίντεο εκτέλεσης του προγράμματος

Ο πλήρης κώδικας μπορεί να βρεθεί στο αρχείο ***ex3.2.8085***

Θεωρητικές Ασκήσεις

3η Άσκηση

α)

Για Q απλό καταχωρητή:

```
SWAP Nible Q
PUSH PSW ;Save Accumulator and Flags
MOV A,Q ;Move contents of Q into A
RRC ;Rotate A right through carry (bit 0 → carry, others shift right)
RRC ;Repeat...
RRC
RRC ;4x RRC = nibble swap
MOV Q,A ;Store result back to Q
POP PSW ;Restore Accumulator and Flags
ENDM
```

Για Q διπλό καταχωρητή:

```
SWAP Nible Q ;Macro to swap high and low nibbles of the byte at address HL
PUSH PSW ;Save accumulator and flags to preserve original state
MOV A,M ;Load value from memory pointed by HL into A
RRC ;Rotate A right (bit 0 → carry, others shift right, carry → bit 7)
RRC
RRC
RRC ;4th rotation → 4 x RRC = rotate right 4 bits = swap nibbles
MOV M,A ;Store swapped value back into memory at HL
POP PSW ;Restore original accumulator and flags
ENDM
```

β)

```
FILL RP,X,K
;Macro to fill memory starting at RP with value K, repeated until B == FFH

PUSH PSW ;Save accumulator and flags
PUSH B ;Save register B (loop counter)
PUSH C ;Save register C (fill value)
PUSH RP ;Save RP register pair

MVI C,K ;Load the value to fill into C
MVI B,X ;Load initial counter value into B

START:
MOV A,C ;Move fill value to A
MOV M,A ;Store it at the address pointed by RP
INX RP ;Move to next memory address
INC B ;Increment counter
MOV A,B ;Copy B to A for comparison
CPI FFH ;Compare A to FFH
JNZ START ;If not FFH, continue looping

POP RP ;Restore RP
POP C ;Restore C
POP B ;Restore B
POP PSW ;Restore A and flags
ENDM
```

γ)

```
RHLR: ;Rotate HL register pair right
    PUSH PSW ;Save PSW (Accumulator and flags)
    MOV A,H ;Load H (high byte) into A
    RRC ;Rotate A right (carry to bit 7)
    MOV A,L ;Load L (low byte) into A
    RAR ;Rotate A right through carry
    MOV L,A ;Store result back in L
    MOV A,H ;Reload H into A
    RAR ;Rotate A right through carry again
    MOV H,A ;Store result back in H
    POP PSW ;Restore PSW
ENDM
```

4η Άσκηση

- Αρχική κατάσταση:

Ο PC δείχνει στη διεύθυνση 0840H, η οποία περιέχει το CD που αποτελεί και τον κωδικό της εντολής CALL που είναι CD και τη διεύθυνση 9000H ανάποδα στις επόμενες θέσεις της μνήμης.

Address	Content
0840H	CD
0841H	00
0842H	09
0843H	//Επόμενη εντολή
…	…
0900H	//Εντολή στην οποία μας στέλνει η CALL
0901H	…
XXXXH	C9

Ταυτόχρονα, **ο SP δείχνει στη διεύθυνση 3000H.**

- Η εντολή CALL 0900H θα εκτελεστεί και θα συμβούν τα παρακάτω:

[(SP)-1]←(PCH)

[(SP)-2]←(PCL)

(SP)←(SP)-2

(PC)←(adr)

Συνεπώς, στο Stack θα αποθηκευτεί το PC μετά την εκτέλεση της CALL, που σημαίνει ότι στον Stack Pointer θα αποθηκευτεί η τιμή διεύθυνσης **0843H** και πλέον θα δείχνει 2 θέσεις πριν δηλαδή στη διεύθυνση **2FFEH** όπου θα αποθηκευτεί το Low Address του PC (43) ενώ πρώτα θα έχει περάσει από την διεύθυνση 2FFFH και θα έχει αποθηκεύσει εκεί την High PC Address (08).

Ταυτόχρονα το νέο PC θα δείχνει στο **0900H.**

Συνεπώς η νέα κατάσταση θα είναι η εξής για το SP:

Address	Content
2FFEH	43
2FFFH	08
3000H	…

Για το PC:

Address	Content
0840H	CD
0841H	00
0842H	09
0843H	//Επόμενη εντολή
…	…
0900H	//Εντολή στην οποία μας στέλνει η CALL
0901H	…
XXXXH	C9

- Η διακοπή RST 5.5 προκαλεί τα παρακάτω:

[(SP)-1]←(PCH)

[(SP)-2]←(PCL)

(SP)←(SP)-2

(PC)←8n

Τώρα στον Stack Pointer θα αποθηκευτεί όπως πριν σε 2 θέσεις πιο πάνω το Low και High PC Address (**0900H**):

Address	Content
2FFCH	00
2FFDH	09
2FFEH	43
2FFFH	08
3000H	…

Γνωρίζουμε ότι για RST n, το PC πάει στη διεύθυνση 8n, οπότε **8*5.5=44=2C** άρα μεταβαίνει στην **002CH,** για να μεταβεί στην ρουτίνα εξυπηρέτησης:

Address	Content
002CH	//Ρουτίνα εξυπηρέτησης RST 5.5
002DH	…
…	…
0840H	CD
0841H	00
0842H	09
0843H	//Επόμενη εντολή
…	…
0900H	//Εντολή στην οποία μας στέλνει η CALL
0901H	…
XXXXH	C9

- Μετά το τέλος της ρουτίνας εξυπηρέτησης:

Από το Stack θα επαναφερθεί το PC στην **0900H** και θα έχουμε την παρακάτω εικόνα για τον SP:

Address	Content
2FFCH	00
2FFDH	09
2FFEH	43
2FFFH	08
3000H	…

Για το PC:

Address	Content
0840H	CD
0841H	00
0842H	09
0843H	//Επόμενη εντολή
…	…
0900H	//Εντολές όπου μας στέλνει η CALL
0901H	…
XXXXH	C9

- Μόλις έρθει η εντολή **RET (C9)**:

$$(PCL) \leftarrow [(SP)]$$

$$(PCH) \leftarrow [(SP)+1]$$

$$(SP) \leftarrow (SP)+2$$

Το Stack επιστρέφει την αρχική του θέση **3000H** και επιστρέφει στο PC την αρχική του τιμή **0843H** για να συνεχίσει την κανονική ροή του προγράμματος και να εκτελέσει ότι υπήρχε μετά το CALL 0900H.

Συνεπώς, το περιεχόμενο του stack:

Address	Content
2FFCH	00
2FFDH	09
2FFEH	43
2FFFH	08
3000H	…

Και για το PC:

Address	Content
0840H	CD
0841H	00
0842H	09
0843H	//Επόμενη εντολή
…	…
0900H	//Εντολές όπου μας στέλνει η CALL
0901H	…
XXXXH	C9

5η Άσκηση

α)

```
MVI A,0DH ;Enable interrupts via SIM (0D=00001101b: enable RST 6.5)
MVI B,00H ;B = 0 → counter for the number of 4-bit nibbles received (up to 64)
LXI D,0000H ;DE = 0 → 16-bit register pair to store the total sum
SIM ;Set interrupt mask (enable/disable RST 5.5, 6.5, 7.5 per A)
EI

START:
MOV A,B ;Check how many 4-bit values have been received so far
```

```
CPI 40H ;Compare with 40h (64 decimal) → since 32 bytes = 64 nibbles
JNZ START ;If not done receiving, loop
JMP MEAN ;If all 32 bytes received, calculate average

;INTERRUPT ROUTINE (Triggered on each 4-bit nibble received)
INTR_ROUTINE:
INR B ;Increase nibble counter
MOV A,B
RRC ;Rotate right → check if this is LSB (odd B) or MSB (even B)
IN 20H ;Read 4-bit data from port 20H (only lower nibble is valid)

JNC SEC ;If Carry = 0 → MSB nibble → jump to SEC
;Else → LSB nibble: combine with MSB already shifted earlier
ADD E ;Add LSB to the low byte of the sum
MOV E,A ;Store result in E
JNC RETE ;If no carry occurred, no need to increment high byte
INR D ;Carry occurred → increment high byte (D)

RETE:
RET ;Return from interrupt

SEC: ;Process MSB (shift left 4 bits)
RRC ;Rotate right (x4) = equivalent of shifting left 4 bits
RRC
RRC
RRC
ADD E ;Add to the current sum
MOV E,A
JNC RETE ;If no carry to D, skip increment
INR D ;Carry → increment D
RET

;MEAN: compute average of 32 values stored in DE
MEAN:
CALL RR ;Shift right DE by 1 (divide by 2)
CALL RR ;Again → divide by 2^2
CALL RR ;Divide by 2^3
CALL RR ;Divide by 2^4
CALL RR ;Divide by 2^5 (total: divide by 32)
JMP ENDD ;Done

;RR: Right shift DE register pair by 1 bit
```

```
RR:
PUSH PSW ;Save accumulator and flags
STC ;Set carry to ensure known state
CMC ;Complement carry → now cleared
MOV A,D ;Load high byte
RAR ;Rotate right through carry → MSB of D goes into carry
MOV D,A ;Save shifted result
MOV A,E ;Load low byte
RAR ;Rotate right through carry → LSB shifts in carry from D
MOV E,A ;Save shifted result
POP PSW ;Restore A and flags
RET

ENDD:
END
```

β)

```
MVI A,0DH ;Initialize Accumulator with 0DH
MVI B,00H ;Initialize B register to 00H
LXI D,0000H ;Initialize D-E pair to 0000H

START:
IN 20H ;Read data from I/O port 20H
RLC ;Rotate left the Accumulator (Shift left with carry)
JNC START ;If no carry, jump to START
RRC ;Rotate right the Accumulator (Shift right with carry)
MOV C,A ;Move Accumulator to C
MOV A,B ;Load Accumulator with B
CPI 40H ;Compare A with 40H
JNZ START ;If not equal, jump to START
JMP MEAN ;Otherwise, jump to MEAN

START2:
IN 20H ;Read data from I/O port 20H again
RLC ;Rotate left the Accumulator
JC START2 ;If carry set, continue looping
JMP START ;Jump to START

INTR_ROUTINE:
```

```
INR B ;Increment B register
MOV A,B ;Move B to Accumulator
RRC ;Rotate right Accumulator
MOV A,C ;Move C to Accumulator
JNC SEC ;If no carry, jump to SEC
ADD E ;Add E to Accumulator
MOV E,A ;Move Accumulator to E
JNC START2 ;If no carry, jump to START2
INR D ;Increment D register

RETE:
JMP START2 ;Jump to START2

SEC:
RRC ;Rotate right Accumulator
RRC
RRC
RRC
ADD E ;Add E to Accumulator
MOV E,A ;Move Accumulator to E
JNC START2 ;If no carry, jump to START2
INR D ;Increment D register
JMP START2 ;Jump to START2

MEAN:
CALL RR ;Call RR routine 5 times
CALL RR
CALL RR
CALL RR
CALL RR
JMP ENDD

RR:
PUSH PSW ;Save PSW
STC ;Set carry flag
CMC ;Complement carry flag
MOV A,D ;Move D to Accumulator
RAR ;Rotate right Accumulator through carry
MOV D,A ;Move Accumulator to D
MOV A,E ;Move E to Accumulator
RAR ;Rotate right Accumulator through carry
MOV E,A ;Move Accumulator to E
```

```
POP PSW
RET

ENDD:
END
```